

SOFTWARE ENGINEERING

CS-UH 2012, SPRING 2025

Modeling Framework Activity:

Design Patterns



Prof. Mai Oudah
mai.oudah@nyu.edu

OUTLINE

- Design to Code
- Design Patterns
 - Creational Design Patterns
 - Structural Design Patterns
 - Behavioural Design Patterns

DESIGN TO CODE

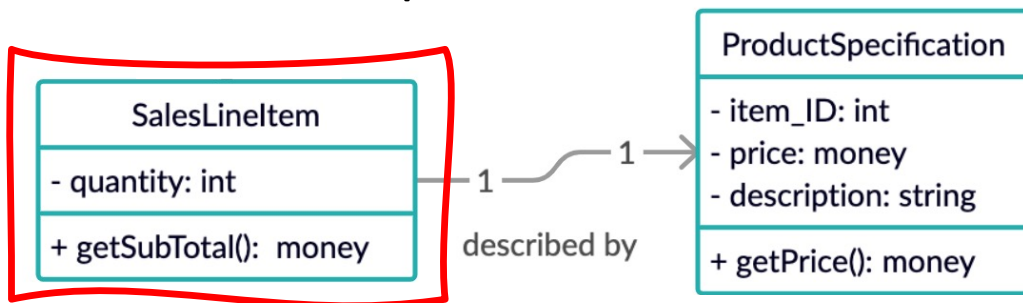
- From design, we can derive:
 - Class definitions
 - Method definitions



DESIGN TO CODE

Class Definitions

- In the class diagram, the attributes and method signatures of a class can be defined in detail
- Example:



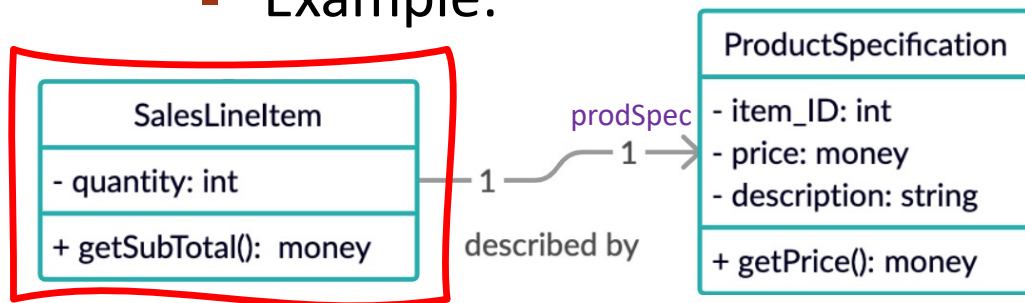
//Java implementation of the **SalesLineItem** Class:

```
public class SalesLineItem
{
    private int quantity; //the class attribute
    //the constructor method for attribute initialization
    public SalesLineItem(ProductSpecification spec, int qty)
    {...}
    //the getSubTotal() method
    public money getSubTotal()
    {...}
}
```

DESIGN TO CODE

Class Definitions – Reference attributes/arguments

- An attribute/argument that refers to another object
- This can be derived from **the associations** in the class diagram
- Example:



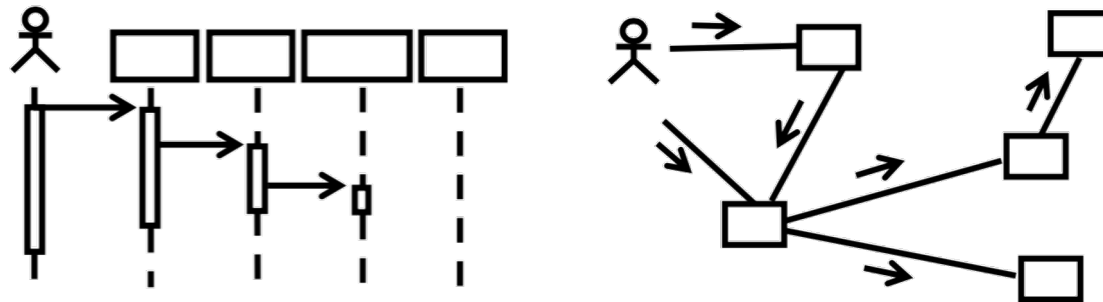
//Java implementation:

```
public class SalesLineItem
{
    private int quantity; //the class attribute
    private ProductSpecification prodSpec; //the ref. attr.
    //the constructor method for attribute initialization
    public SalesLineItem(ProductSpecification spec, int qty)
    {...}
    //the getSubTotal() method
    public money getSubTotal()
    {...}
}
```

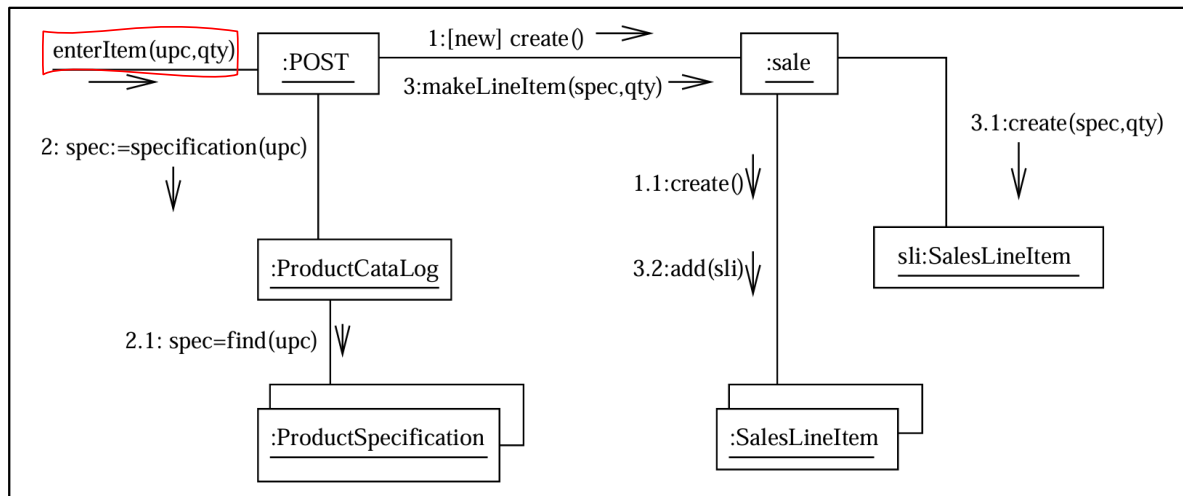
DESIGN TO CODE

Method Definitions

- Some method structures can be extracted from the different interaction diagrams.
- The sequence of the messages translates to a series of statements in a method definition.



EXAMPLE – METHOD DEFINITION



//Java implementation of the enterItem() method in POST class:

```

public void enterItem(int upc, int qty)
{   if (isNewSale()) { sale = new Sale(); //msg 1 ... you need to define isNewSale()
    method outside enterItem() to check if the sale is new }

```

```

    ProductSpecification spec = prodCatalog.specification(upc); //msg 2

```

```

    sale.makeLineItem(spec, qty); //msg 3

```

```

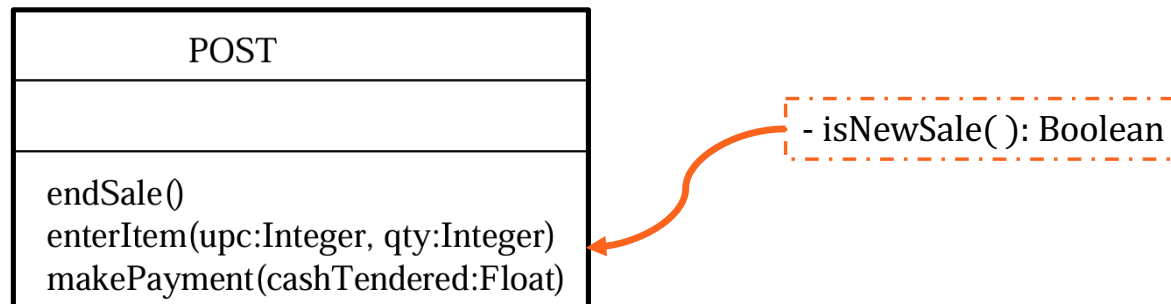
}

```

DESIGN TO CODE

Updating Class Definitions

- Whenever new methods or attributes are discovered during coding, the existing UML models have to be updated to maintain consistency.
- Example:

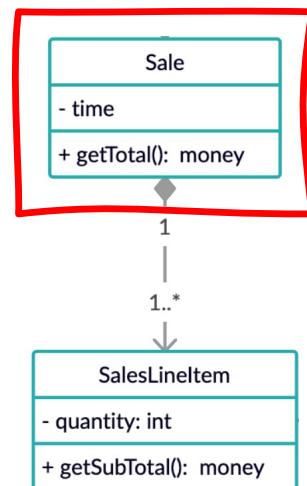


DESIGN TO CODE

Container Classes

- In order to maintain visibility to a group of other objects; the need for this is usually evident from the association multiplicity value in a class diagram.

- Example:

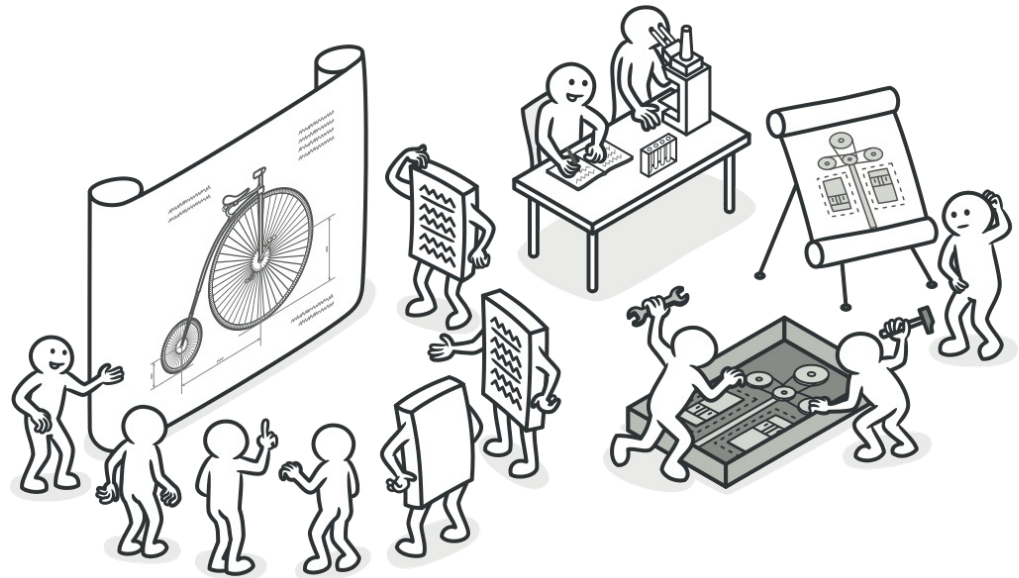


//Java implementation of the **Sale** Class:

```
public class Sale
{
    private Time time; //the class attribute
    private Vector lineItems; // a container attribute
    ...
    //the getTotal() method
    public money getTotal()
    {...}
}
```

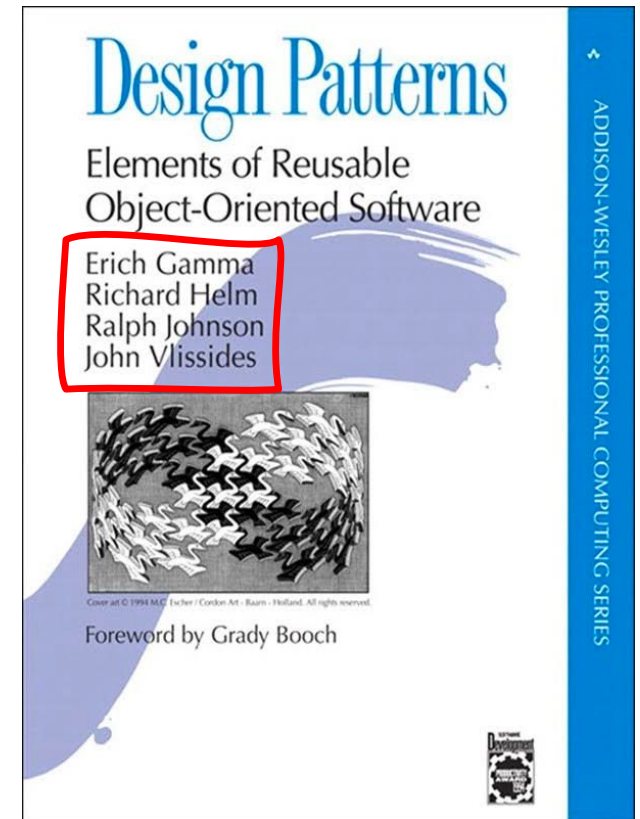
DESIGN PATTERNS

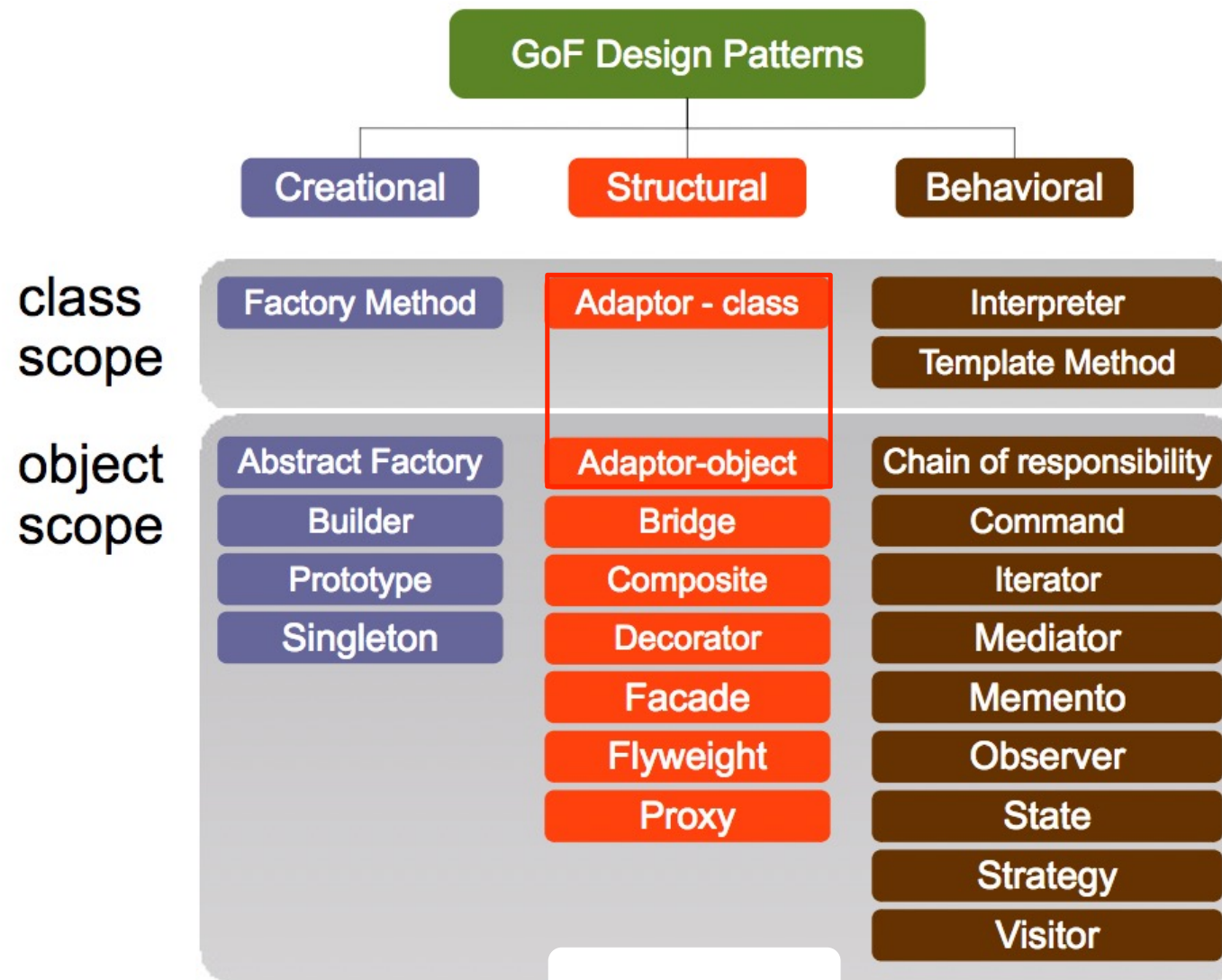
- A **design pattern** is a standard **reusable solution** to a commonly occurring problem/scenario in software design



GoF DESIGN PATTERNS

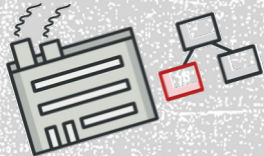
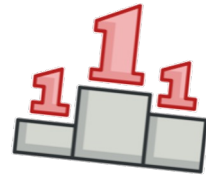
- The **Gang of Four** are the four authors who sat the foundation for all design patterns
- Classified by **purpose**:
 - Creational
 - Structural
 - Behavioral
- Classified by **scope**:
 - Class scope (static)
 - Object scope (dynamic)







CREATIONAL PATTERNS



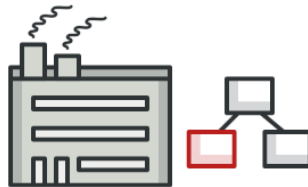
CREATIONAL PATTERNS

- Encapsulate the **creation process**
 - make an application independent of how its objects are created
 - Conceal how objects are created and put together from other components within the system

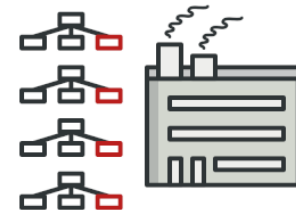
CREATIONAL PATTERNS



Singleton



Factory
Method

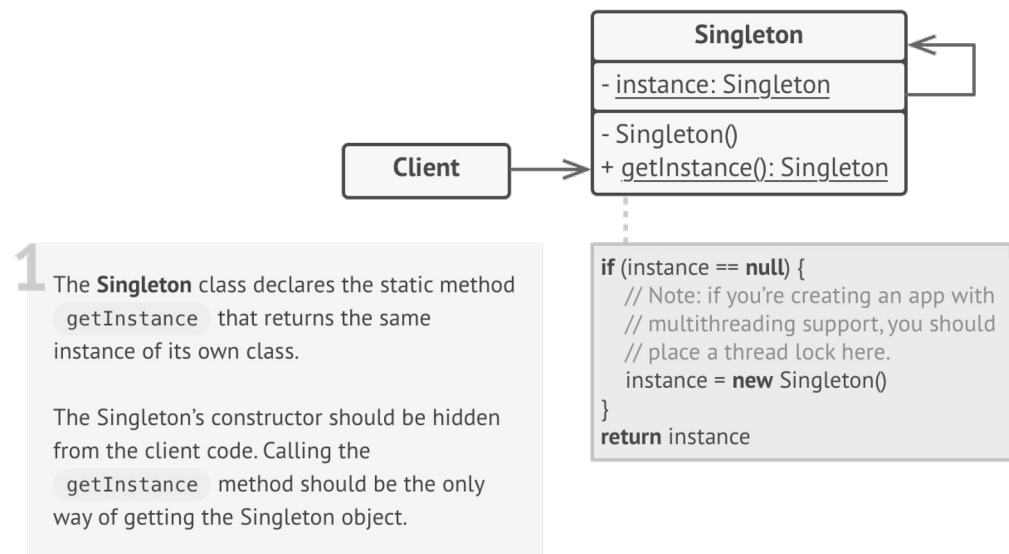


Abstract
Factory

CREATIONAL PATTERNS

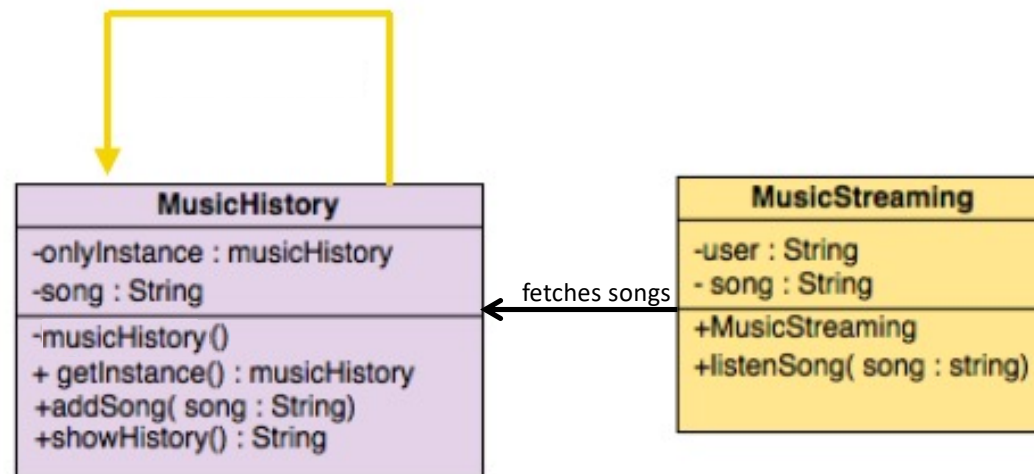
SINGLETON

- It ensures that a class has **only one object**, while providing a **global access point** to that object.



CREATIONAL PATTERNS

SINGLETON: *EXAMPLE*



CREATIONAL PATTERNS

SINGLETON: HOW TO IMPLEMENT

1. Add a private static field to the class for storing the singleton instance.
2. Declare a public static creation method for getting the singleton instance
3. Implement “lazy initialization” inside the static method. It should create a new object on its first call and put it into the static field. The method should always return that instance on all subsequent calls.
4. Make the constructor of the class private. The static method of the class will still be able to call the constructor, but not the other objects.
5. All calls should be redirected/replaced from the singleton’s constructor to its static creation method.